05 · PHC dashboard and the catchment rule

The health worker's home page, and the one rule every worker page depends on: you only see your own citizens.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

The PHC dashboard shows the facility name, four headline numbers (people needing review, visits needing an outcome, citizens in the catchment, readings in the last 7 days), the worst 6 people to review today with a severity filter, and 8-week mini charts.

Every worker query goes through `services/catchment.py`: a worker sees only citizens with the same `phc_id` as theirs. Someone outside the catchment gets exactly the same 404 as someone who does not exist. A worker who is not linked to any centre gets a 409 and a clear "No health centre linked" page instead of an empty dashboard.

Files involved

Only the files this feature uses.

```
backend/app/
├── services/catchment.py       catchment_ids(), get_catchment_citizen(),
│                               require_catchment_worker()
├── api/routes/phc.py           GET /phc/dashboard, GET /phc/overview, helper queries (_open_flag,
│                               _severity_sql)
└── services/triage.py          BANDS: the severity thresholds
frontend/src/
├── pages/phc/PhcDashboard.tsx  the page and KpiCard (the weekly mini chart)
├── components/app/phc.tsx      workerState(), NoFacility, CitizenWorkCard
└── services/phc.ts             getDashboard(), getOverview(), getWorklist()
```

How the pieces connect

1. Every worker route lists `worker: User = Depends(require_catchment_worker)`; that runs first and stops with 409 if the worker has no centre.
2. The page loads three things in parallel: `/phc/dashboard` (counts), `/phc/overview` (weekly trends) and `/phc/worklist` (people to review, guide 06).
3. `workerState(dash)` shows the right screen for a 409 or an error; otherwise the dashboard renders.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/services/catchment.py` · catchment_ids

```
32  def catchment_ids(worker: User) -> Select[tuple[uuid.UUID]]:
33      """Ids of the catchment, for `.in_()` filters on other tables."""
34      if worker.phc_id is None:
35          return select(User.id).where(false())
36      return select(User.id).where(User.phc_id == worker.phc_id, User.role == UserRole.user)
```

What the code means

- Returns a **query**, not a list: other queries use it inside `.in_(...)`, so the database does the filtering in one go.
- `select(User.id).where(false())` matches nothing: a worker without a centre serves **nobody**, never everybody.
- `User.role == UserRole.user` keeps other workers out of the list.

2. `backend/app/services/catchment.py` · require_catchment_worker

```
55  async def require_catchment_worker(worker: User = Depends(require_phc_worker)) -> User:
56      """An approved worker linked to a facility. 409 otherwise: "0 flagged" would read as a quiet day."""
57      if worker.phc_id is None:
58          raise HTTPException(status.HTTP_409_CONFLICT, NO_FACILITY)
59      return worker
```

What the code means

- A dependency that builds on another one: `Depends(require_phc_worker)` first checks the role, then this checks the centre link.
- 409 Conflict is used on purpose: an empty dashboard would read as "a quiet day" and be trusted.

3. `backend/app/services/catchment.py` · get_catchment_citizen

```
47  async def get_catchment_citizen(session: AsyncSession, worker: User, user_id: uuid.UUID) -> User:
48      """The citizen, or 404 with the same body whether they do not exist or are not this worker's."""
49      citizen = await session.get(User, user_id)
50      if citizen is None or not in_catchment(worker, citizen):
51          raise citizen_not_found()
52      return citizen
```

What the code means

- `await session.get(User, user_id)` loads one row by its id.
- "Does not exist" and "not yours" raise the same error with the same text, so a worker cannot discover ids outside their area.

4. `backend/app/api/routes/phc.py` · _open_flag

```
95  def _open_flag():
96      return (ProcessedData.is_anomaly.is_(True), ProcessedData.resolution.is_(None))
```

What the code means

- Returns a pair of conditions. Callers write `.where(..., *_open_flag())`; the `*` spreads the pair into two arguments.
- Open flag = flagged and not yet resolved. Defined once so every page counts the same way.

5. `backend/app/api/routes/phc.py` · _severity_sql

```
 99  def _severity_sql():
100      """The triage bands as a SQL CASE, built from triage.BANDS so the two cannot drift."""
101      return case(*[(ProcessedData.anomaly_score >= floor, name) for name, floor in BANDS[:-1]], else_=BANDS[-
             1][0])
```

What the code means

- `case(...)` builds an SQL CASE expression, so the database can group flags by severity.
- It is generated from `triage.BANDS` (`BANDS[:-1]` is every band except the last; `else_` gives the last), so Python and SQL can never disagree.

6. `backend/app/api/routes/phc.py` · dashboard

```
184 async def dashboard(worker: User = Depends(require_catchment_worker), db: AsyncSession = Depends(get_session
        )) -> DashboardOut:
185     now = now_utc()
186     members = catchment_ids(worker)
187     facility = await db.get(ValidPhcFacility, worker.phc_id)
188     assert facility is not None
189 
190     severity = _severity_sql()
191     by_severity = dict(
192         (
193             await db.execute(
194                 select(severity, func.count()).where(ProcessedData.user_id.in_(members), *_open_flag()).grou
                        p_by(severity)
195             )
196         ).all()
197     )
198     flagged = await db.scalar(
199         select(func.count(func.distinct(ProcessedData.user_id))).where(ProcessedData.user_id.in_(members), *
                _open_flag())
200     )
201     week = now - timedelta(days=7)
202     readings_7d, submitting_7d = (
203         await db.execute(
204             select(func.count(), func.count(func.distinct(HealthRecord.user_id))).where(
205                 HealthRecord.user_id.in_(members), HealthRecord.timestamp >= week
206             )
207         )
208     ).one()
209     tomorrow = local_day_start(local_date(now) + timedelta(days=1))
210     my_visits = select(ScheduledVisit.id).where(
211         ScheduledVisit.worker_id == worker.id,
212         ScheduledVisit.citizen_id.in_(members),
213         ScheduledVisit.status == VisitStatus.scheduled,
214     )
215     return DashboardOut(
216         facility=_facility_out(facility),
217         citizens=await _count(db, catchment_citizens(worker)),
218         citizens_flagged=flagged or 0,
219         open_flags=sum(by_severity.values()),
220         open_by_severity=SeverityCounts(**by_severity),
221         readings_last_7_days=readings_7d,
222         citizens_submitting_last_7_days=submitting_7d,
223         engagement=await _engagement_counts(db, worker, now),
224         silent_after_days=SILENT_AFTER.days,
225         visits_needing_outcome=await _count(db, my_visits.where(ScheduledVisit.scheduled_for < overdue_befor
                e(now))),
226         visits_today=await _count(
227             db, my_visits.where(ScheduledVisit.scheduled_for > now, ScheduledVisit.scheduled_for < tomorrow)
```

What the code means

- `dict(...)` of `(severity, count)` pairs gives open flags per severity.
- `func.count(func.distinct(ProcessedData.user_id))` counts people with open flags.
- `readings_7d, submitting_7d = (...).one()` reads two numbers from a single query: total readings and distinct citizens in the last 7 days.
- `my_visits` is a reusable query of this worker's open visits; it is narrowed twice later for "needs outcome" and "today".

7. `frontend/src/components/app/phc.tsx` · workerState

```
35  export function workerState(r: { error: string | null; status: number | null; data: unknown; reload: () => v
        oid }) {
36    if (r.status === 409 && r.error) return <NoFacility message={r.error} />;
37    if (r.error && !r.data) return <ErrorNote message={r.error} onRetry={r.reload} />;
38    return null;
39  }
```

What the code means

- Returns a screen to show instead of the page, or `null` when the page may render. Every worker page starts with `const blocked = workerState(x); if (blocked) return blocked;`.
- `r.status === 409` is the "no centre linked" case from `require_catchment_worker`.

8. `frontend/src/pages/phc/PhcDashboard.tsx` · PhcDashboard (data part)

```
59  export function PhcDashboard() {
60    const { params, set } = useQueryState();
61    const severity = oneOf(params.get("severity"), SEVERITIES, null);
62    const dash = useResource((s) => getDashboard(s), "dashboard");
63    const overview = useResource((s) => getOverview(s), "overview");
64    const work = useResource((s) => getWorklist({}, s), "worklist");
65
66    const people: WorkItem[] = useMemo(() => work.data?.items ?? [], [work.data]);
67    const counts = useMemo(() => {
68      const out = { critical: 0, high: 0, moderate: 0, low: 0 } as Record<Severity, number>;
69      for (const p of people) out[p.worst_severity] += 1;
70      return out;
71    }, [people]);
72    const filtered = severity ? people.filter((p) => p.worst_severity === severity) : people;
73    const shown = filtered.slice(0, PEOPLE_SHOWN);
74
75    const blocked = workerState(dash);
76    if (blocked) return blocked;
77    if (!dash.data) return <Loading label="Loading your catchment" />;
78    const d = dash.data;
79
80    const refresh = () => {
81      work.reload();
82      dash.reload();
83    };
84
```

What the code means

- `oneOf(params.get("severity"), SEVERITIES, null)` reads the severity tab from the URL safely.
- Three `useResource` calls load in parallel; each part of the page shows its own loading or error state.
- `useMemo` counts people per worst severity for the tab labels; `filtered.slice(0, PEOPLE_SHOWN)` keeps the top 6.
- `refresh` reloads the worklist and the counts after a worker resolves someone.

9. `frontend/src/pages/phc/PhcDashboard.tsx` · KpiCard

```
16  function KpiCard({ kpi }: { kpi: Kpi }) {
17    const complete = kpi.weeks.filter((w) => !w.partial);
18    const last = complete[complete.length - 1];
19    const current = kpi.weeks[kpi.weeks.length - 1];
20    return (
21      <Card>
22        <p className="text-xs font-medium uppercase tracking-wide text-ink-soft">{kpi.label}</p>
23        <p className="mt-1 text-2xl font-semibold tabular-nums text-ink">{last?.value ?? 0}</p>
24        <p className="text-xs text-ink-soft">
25          last full week · {current.value} so far this week
26        </p>
27        <div className="mt-3 h-16" aria-hidden="true">
28          <ResponsiveContainer width="100%" height="100%">
29            <BarChart accessibilityLayer={false} data={kpi.weeks} margin={{ top: 0, right: 0, bottom: 0, left:
                  0 }} barCategoryGap={2}>
    ... (28 more lines in the file: the Recharts bar chart follows)
```

What the code means

- `kpi.weeks` has 8 weeks; the current week is marked `partial`. The big number is the last **full** week, so a Monday never looks like a bad week.
- `complete[complete.length - 1]` is the last item of a list; `last?.value ?? 0` falls back to 0.

Syntax used in this feature

Code	What it means
`select(...).where(false())`	A query that matches nothing.
`Depends(other_dependency)`	Dependencies can depend on each other.
`case((cond, value), else_=x)`	SQL CASE WHEN ... THEN ... ELSE ... END.
`f(*pair)`	Spread a tuple into separate arguments.
`(await db.execute(q)).one()`	Exactly one result row.